DOCENTE	NARDULLI MATTIA
UNITA' FORMATIVA	FONDAMENTI DI VERSION CONTROL
ARGOMENTO	INTRODUZIONE AL VERSION CONTROL

Fondamenti di Version Control

Lezione 1 – Corso di Version Control con Git



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

OBIETTIVI

Cosa imparerai in questa lezione

Comprendere il Version Control

Scopriremo cos'è il version control, perché è nato e quali problemi risolve nello sviluppo software moderno

Conoscere Git

Impareremo le basi di Git, lo strumento più utilizzato al mondo per il controllo di versione

Pratica diretta

Creeremo il nostro primo repository e faremo il nostro primo commit, mettendo subito in pratica i concetti

Il version control è una competenza fondamentale per ogni sviluppatore moderno. Che tu lavori da solo o in team, padroneggiare questi strumenti ti permetterà di gestire il codice in modo professionale, tracciare ogni modifica e collaborare efficacemente con altri sviluppatori. Questa lezione getta le fondamenta per diventare un professionista dello sviluppo software.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Cos'è il Version Control?

Il **version control** (o controllo di versione) è un sistema che registra nel tempo le modifiche apportate a uno o più file, permettendoti di recuperare versioni specifiche in un secondo momento. Pensa a un sistema di "macchina del tempo" per il tuo codice.

1

Versioni

Ogni modifica significativa crea una nuova versione del progetto, come un'istantanea nel tempo

2

Storico delle modifiche

Tutte le modifiche vengono registrate con data, autore e descrizione, creando una cronologia completa

3

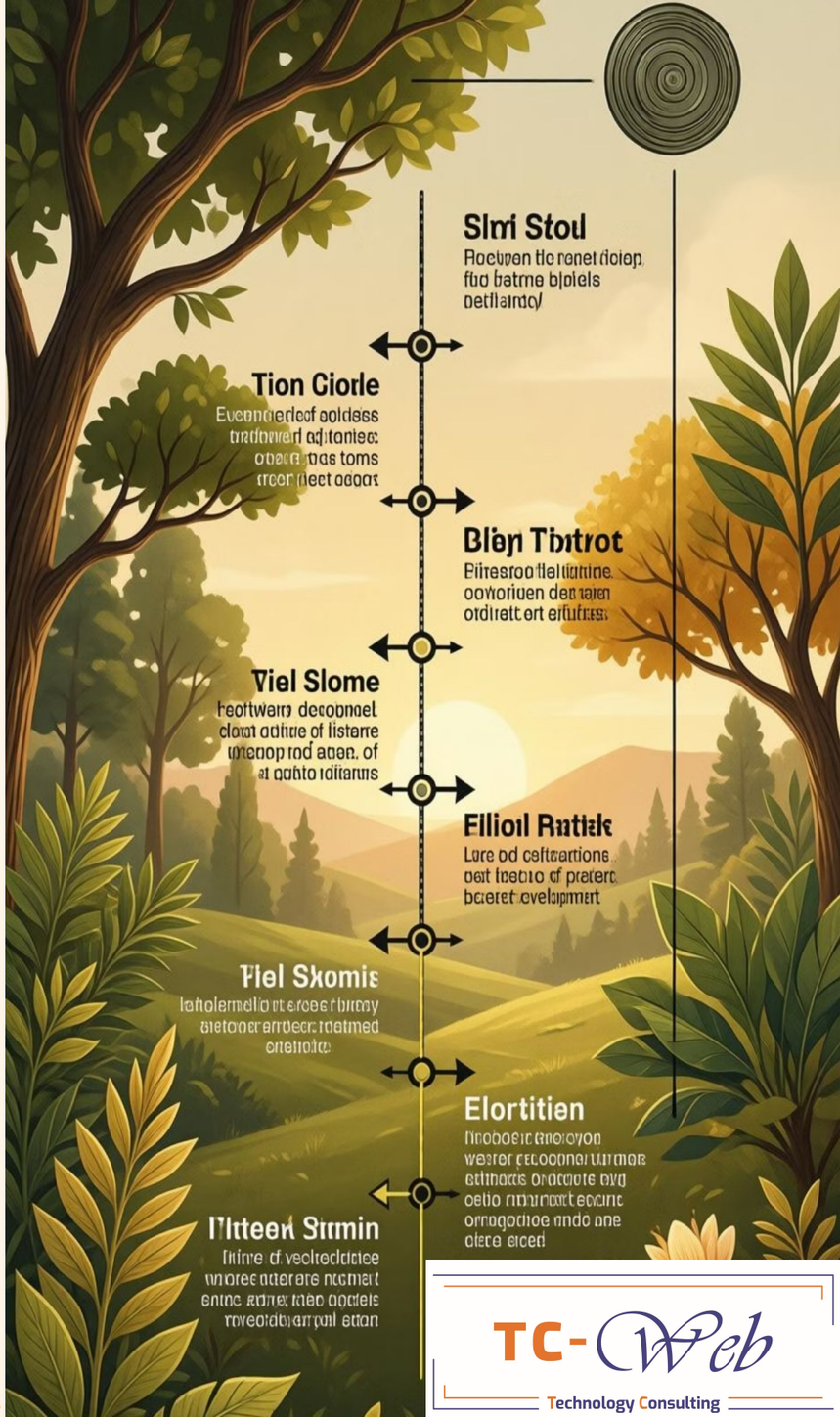
Tracciabilità

Puoi sapere chi ha modificato cosa, quando e perché, rendendo il codice completamente trasparente

4

Collaborazione

Più persone possono lavorare sullo stesso progetto contemporaneamente senza sovrascriversi a vicenda



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Il caos senza Version Control

Scenario familiare?

- `progetto.zip`
- `progetto_v2.zip`
- `progetto_finale.zip`
- `progetto_finale_VERO.zip`
- `progetto_finale_definitivo.zip`
- `progetto_finale_definitivo_v2.zip`
- `progetto_da_consegnare_QUESTO.zip`

Questo caos è il risultato di lavorare **senza un sistema di version control**. Ogni versione è un file separato, non c'è traccia delle modifiche, e trovare "quella modifica di tre settimane fa" diventa un incubo.

Problemi reali

Perdita di lavoro

Sovrascrivere accidentalmente file importanti senza possibilità di recupero

Conflitti nel team

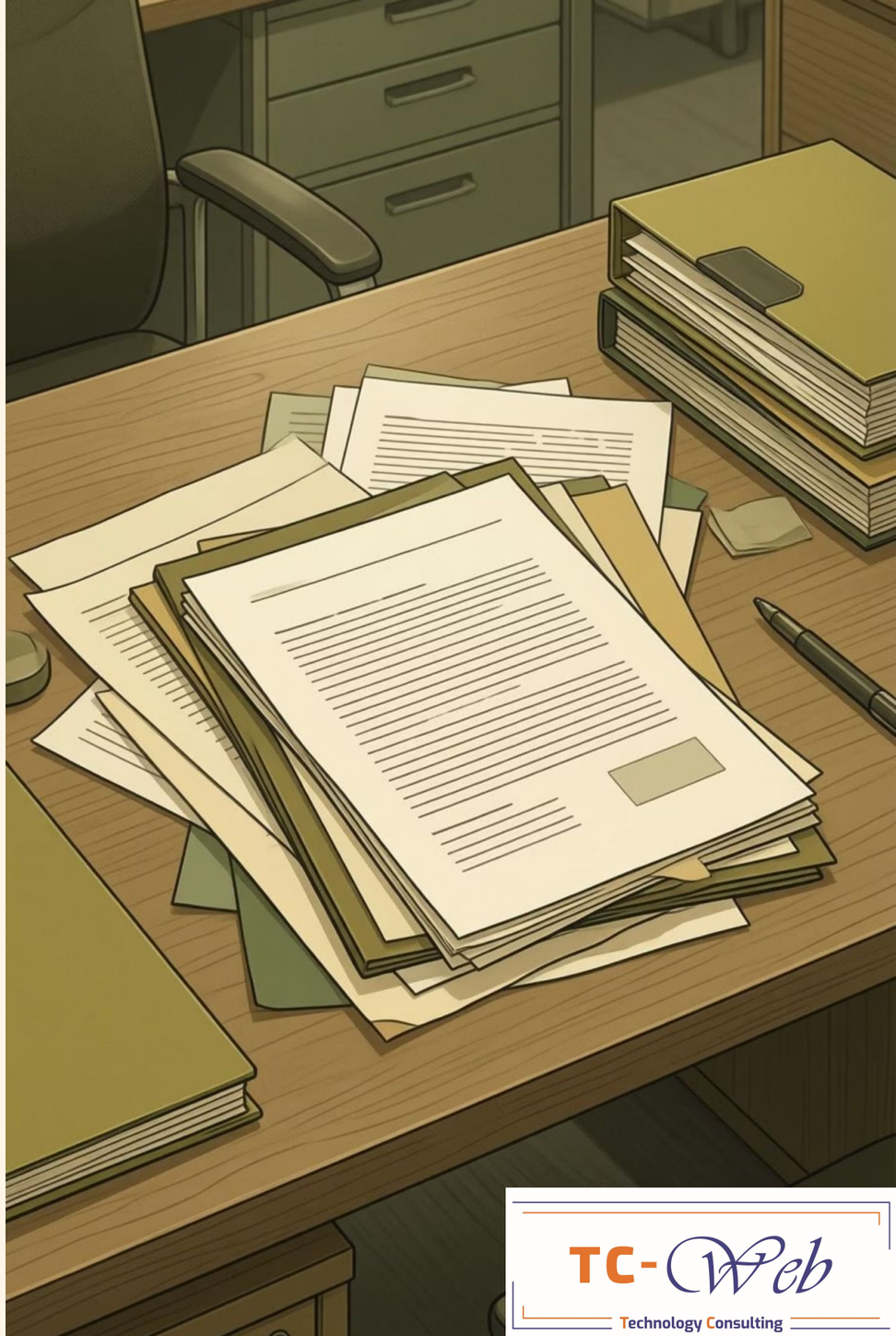
Due persone modificano lo stesso file e una delle due versioni viene persa

Nessuna tracciabilità

Impossibile sapere chi ha fatto cosa, quando e perché una modifica è stata fatta

Backup inefficiente

Duplicare intere cartelle di progetto occupa spazio e crea confusione



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Tre modelli di Version Control

Nel corso degli anni, sono emersi tre principali modelli di version control, ognuno con i propri vantaggi e limitazioni. Comprendere queste differenze aiuta a comprendere perché Git è diventato lo standard industriale.

01

Version Control Locale

Il sistema più semplice: un database locale che tiene traccia delle modifiche ai file sul tuo computer. Esempio: RCS (Revision Control System). **Limite:** funziona solo su un singolo computer, nessuna collaborazione possibile.

02

Version Control Centralizzato

Un unico server centrale contiene tutti i file versionati. I client scaricano le versioni dal server. Esempio: SVN (Subversion), CVS. **Limite:** se il server non è disponibile, nessuno può lavorare o salvare versioni.

03

Version Control Distribuito

Ogni client ha una copia completa dell'intero storico del repository. Esempio: Git, Mercurial. **Vantaggio:** lavoro offline, backup naturale, massima flessibilità nella collaborazione.

Caratteristica	Locale	Centralizzato	Distribuito
Collaborazione	❌ No	✅ Sì	✅✅ Eccellente
Lavoro offline	✅ Sì	❌ Limitato	✅ Completo
Velocità	⚡ Veloce	🐌 Dipende dalla rete	⚡ Velocissimo
Backup	❌ Nessuno	⚠️ Solo sul server	✅ Automatico ovunque

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Dove si usa il Version Control?



Sviluppo Software

L'uso principale: gestire codice sorgente, coordinate feature, risolvere bug e mantenere più versioni del software simultaneamente



Documentazione

Mantenere versioni di manuali, guide utente, documentazione tecnica e specifiche di progetto con tracciabilità completa



Progetti Creativi

Gestire configurazioni di sistema, script di automazione, file di design e qualsiasi progetto che evolve nel tempo

Esempi concreti nel mondo reale

Linux Kernel: migliaia di sviluppatori coordinati via Git
Open Source: progetti come React, Vue.js, TensorFlow su GitHub
Aziende: Google gestisce miliardi di righe di codice con version control
Startup: ogni team di sviluppo moderno usa Git quotidianamente

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Introduzione a Git

Git è un sistema di version control distribuito creato nel 2005 da Linus Torvalds (lo stesso creatore di Linux). È diventato lo standard de facto nell'industria del software grazie alla sua velocità, flessibilità e potenza.

100M+

Repository su GitHub

Oltre 100 milioni di repository ospitati sulla piattaforma più popolare

94%

Adozione professionale

Percentuale di sviluppatori professionisti che usano Git

2005

Anno di nascita

Git è nato per gestire lo sviluppo del kernel Linux

Perché Git è così popolare?



Velocità

Operazioni locali istantanee, nessuna dipendenza dalla rete per la maggior parte delle operazioni



Branching potente

Creare, unire ed eliminare branch è velocissimo e incoraggia la sperimentazione



Completamente distribuito

Ogni clone è un backup completo, puoi lavorare ovunque, anche offline



Integrità dei dati

Ogni file è verificato tramite checksum, impossibile corrompere i dati senza che Git lo sappia

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

CONCETTO CHIAVE

Git non è GitHub

Git: lo strumento locale

Git è un **software di version control** che vive sul tuo computer. Funziona completamente offline e gestisce la storia delle modifiche del tuo codice in modo autonomo.

- Installato localmente sul tuo PC
- Funziona senza connessione Internet
- Gestisce repository in locale
- Strumento a riga di comando

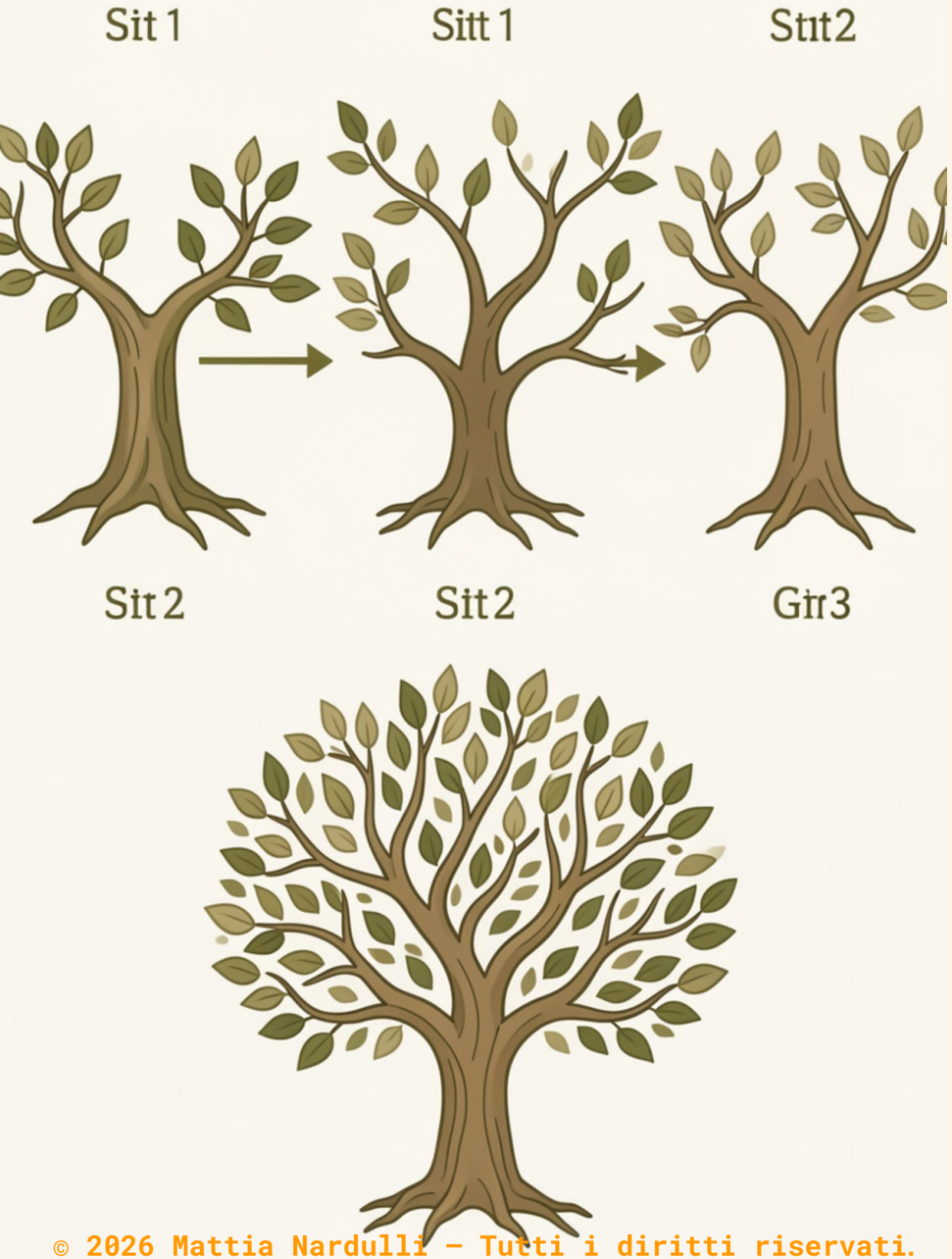
GitHub: la piattaforma online

GitHub è una **piattaforma cloud** che ospita repository Git remoti. Serve per collaborare e condividere il codice con altri sviluppatori.

- Servizio online (richiede Internet)
- Backup remoto dei progetti
- Collaborazione in team
- Interfaccia web visuale

Analogia utile: Git è come Microsoft Word installato sul tuo computer, mentre GitHub è come Google Drive dove salvi e condividi i documenti con altri.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Come funziona Git: il flusso di lavoro

Git non salva automaticamente le modifiche come fa un editor di testo. Segue invece un processo in tre fasi che ti permette di controllare esattamente cosa salvare e quando. Comprendere questo flusso è fondamentale per usare Git efficacemente.

Working Directory

I file su cui stai lavorando attivamente. Le modifiche non sono ancora tracciate da Git.

Staging Area

Area di preparazione dove selezioni le modifiche da includere nel prossimo commit.

```
git add nomefile
```

Repository

Archivio permanente con la storia completa delle modifiche salvate.

```
git commit -m "messaggio"
```

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Il commit: un istantanea del progetto



Cos'è davvero un commit?

Un commit è una **fotografia dello stato del tuo progetto** in un momento specifico. Non è solo un salvataggio: è un punto nella storia del tuo codice a cui puoi sempre tornare.

> "Pensa al commit come a un **checkpoint in un videogioco**: puoi sempre ricaricare da quel punto se qualcosa va storto."

Messaggio descrittivo

Spiega *cosa* è stato modificato e *perché*

Autore e data

Chi ha fatto la modifica e quando

Codice hash univoco

Identificatore unico del commit (es: a3f2b1c)

Snapshot completo

Stato coerente di tutti i file tracciati

Ogni commit ti permette di **viaggiare nel tempo**: puoi vedere com'era il codice in passato, confrontare versioni diverse e capire l'evoluzione del progetto.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Buone pratiche sui commit

Imparare a fare commit efficaci è una competenza fondamentale. Commit ben strutturati rendono la storia del progetto leggibile e facilitano la collaborazione con altri sviluppatori.

Un'idea = un commit

Ogni commit dovrebbe rappresentare una singola modifica logica e completa

Commit frequenti

Meglio tanti commit piccoli che uno grande alla fine della giornata

Messaggi chiari

Descrivi cosa hai fatto in modo che tu (e altri) possiate capirlo tra sei mesi

❌ Esempi da evitare

- `update`
- `fix`
- `modifiche varie`
- `asdasd`
- `funziona!!!`

✅ Esempi corretti

- `Aggiunta validazione form login`
- `Correzione bug nel calcolo totale carrello`
- `Implementato sistema di cache per query`
- `Aggiornata documentazione API utenti`

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Anatomia di un buon messaggio di commit

01

Usa l'imperativo presente

"Aggiunge", "Corregge", "Implementa" - come se stessi dando un ordine a Git

02

Sii specifico ma conciso

Massimo 50-70 caratteri per la prima riga. Vai dritto al punto.

03

Spiega il "perché", non il "cosa"

Il codice mostra cosa hai fatto. Il messaggio spiega perché l'hai fatto.

04

Aggiungi dettagli se necessario

Dopo una riga vuota, puoi aggiungere una descrizione più lunga con contesto addizionale

> **Regola d'oro:** Il tuo messaggio di commit dovrebbe completare la frase "Se applicato, questo commit _____". Esempio: "Se applicato, questo commit *aggiunge validazione form login*".



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Errori comuni quando si inizia con Git

Tutti fanno errori all'inizio. Riconoscere questi pattern ti aiuterà a evitarli e a sviluppare buone abitudini fin da subito.

1

Dimenticare git add

Modifichi i file ma ti dimentichi di aggiungerli alla staging area prima del commit. Risultato: il commit è vuoto o incompleto.

Soluzione: Usa `git status` frequentemente per verificare cosa stai per committare.

2

Fare un solo commit gigante

Lavori tutto il giorno e alla fine fai un unico commit con "modifiche di oggi". Impossibile poi capire cosa è stato fatto.

Soluzione: Committa ogni volta che completi una piccola funzionalità o correzione.

3

Messaggi di commit inutili

Scrivi "fix", "update", "stuff" senza spiegare cosa hai realmente modificato.

Soluzione: Dedica 10 secondi in più per scrivere un messaggio descrittivo significativo.

4

Paura di "rompere tutto"

Hai timore di usare Git perché pensi di poter distruggere il progetto. Questa paura ti blocca.

Soluzione: Git è progettato per essere sicuro. È quasi impossibile perdere davvero del lavoro committato.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Git è fatto per sperimentare in sicurezza

Perché non devi avere paura

Una delle cose più difficili da capire per i principianti è che **Git è progettato proprio per permetterti di sbagliare**. È un sistema di sicurezza, non un campo minato.

- ### Ogni commit è permanente
  Una volta committato qualcosa, è quasi impossibile perderlo davvero
- ### Puoi sempre tornare indietro
  Git ti permette di annullare, modificare o ripristinare qualsiasi cambiamento
- ### Gli esperimenti sono protetti
  I branch (che vedrai nelle prossime lezioni) ti permettono di provare cose nuove senza rischi



Mentalità corretta

"Sbagliare è parte del processo di apprendimento. Git è la tua rete di sicurezza."

Consiglio pratico: Crea una cartella di test e sperimenta liberamente con Git. Prova a fare commit, a modificare file, a vedere la history. Non c'è modo migliore per imparare che provare direttamente.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Installazione e configurazione di Git

Installazione per sistema operativo

🪟 Windows

Scarica Git per Windows da **git-scm.com** e segui l'installer. Verrà installato anche Git Bash, un terminale Unix-like per Windows.

🍎 macOS

Installa tramite Homebrew con `brew install git` oppure scarica l'installer dal sito ufficiale. Git è già incluso negli Xcode Command Line Tools.

🐧 Linux

Usa il package manager della tua distribuzione: `apt-get install git` (Debian/Ubuntu) o `yum install git` (Red Hat/Fedora).

Configurazione iniziale

Dopo l'installazione, è fondamentale configurare la tua identità.

Queste informazioni verranno associate ad ogni commit che farai.

```
git config --global user.name "Mario Rossi"
git config --global user.email
"mario.rossi@example.com"
```

Per verificare la configurazione:

```
git config --list
```

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Il nostro primo repository Git

È arrivato il momento di mettere in pratica ciò che abbiamo imparato. Creeremo insieme il nostro primo repository Git e faremo il nostro primo commit. Segui attentamente i passaggi.

Crea una cartella di progetto

Apri il terminale e crea una nuova cartella: `mkdir mio-primo-progetto` poi spostati al suo interno: `cd mio-primo-progetto`

Inizializza il repository

Esegui il comando `git init` per inizializzare un nuovo repository Git nella cartella corrente. Vedrai il messaggio "Initialized empty Git repository"

Crea un file

Crea un file di testo: `echo "Il mio primo progetto Git" > README.md` oppure usa il tuo editor preferito per creare il file

Aggiungi alla staging area

Prepara il file per il commit: `git add README.md`. Questo sposta il file nell'area di staging, pronto per essere salvato

Fai il primo commit

Salva la modifica nello storico: `git commit -m "Primo commit: aggiunto README"`. Congratulazioni, hai creato il tuo primo commit!

Obiettivi dell'esercitazione

- Familiarizzare con i comandi base di Git
- Comprendere il flusso: working directory → staging area → repository
- Vedere concretamente come Git traccia le modifiche
- Acquisire confidenza con il terminale

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Esercizio in autonomia

Istruzioni Passo dopo Passo

1. Preparazione:
   - Apri il terminale del tuo computer.
   - Crea una cartella chiamata diario-git e spostati al suo interno.
   - *Suggerimento*: Usa i comandi mkdir e cd.
2. Inizializzazione:
   - Trasforma questa semplice cartella in un repository Git.
   - *Verifica*: Se esegui git status, dovresti vedere che Git è attivo (anche se non ci sono ancora file).
3. Creazione del Contenuto:
   - Crea un file chiamato appunti.txt.
   - Scrivi all'interno del file una frase che descriva il comando Git che ti sembra più utile (es: "Il comando git init serve a iniziare tutto!"). Puoi usare un editor di testo o il comando echo.
4. Tracciamento (Staging Area):
   - Aggiungi il file appunti.txt alla "Staging Area".
   - *Verifica*: Controlla di nuovo con git status. Il file dovrebbe apparire in verde (pronto per il commit).
5. Il Primo "Salvataggio" (Commit):
   - Esegui il commit del file con un messaggio chiaro, ad esempio: "Aggiunto il primo appunto sul diario".
6. La Modifica:
   - Modifica il file appunti.txt aggiungendo una seconda riga con la data di oggi.
   - Esegui di nuovo i passaggi di Add e Commit per salvare questa seconda versione.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Comandi fondamentali

Prima di passare alla prossima lezione, assicurati di aver compreso e memorizzato questi comandi essenziali. Li userai costantemente nel tuo lavoro quotidiano con Git.

`git status`

Mostra lo stato attuale: quali file sono modificati, in staging, o non tracciati

Quando usarlo: Sempre, prima di ogni operazione. È il tuo migliore amico.

`git add nomefile`

Aggiunge un file specifico alla staging area, pronto per essere committato

Variante: `git add .` aggiunge tutti i file modificati (usa con cautela)

`git commit -m "messaggio"`

Crea un commit con i file in staging, salvando un'istantanea del progetto

Ricorda: Il messaggio deve essere chiaro e descrittivo

`git log`

Visualizza la cronologia dei commit del repository con autori, date e messaggi

Variante: `git log --oneline` per una vista compatta

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

GRAZIE A TUTTI…..

Via Schiaparelli 14 - 10148 Torino

https://www.tc-web.it

Mail: franco.grivet@tc-web.it

© 2026 Mattia Nardulli – Tutti i diritti riservati.